PayTrace

Technical Design Document: order-level payment reconciliation for a D2C brand

Author	Ayush Narayan
Code	https://github.com/marvelayush/paytrace
Live app	https://paytrace-two.vercel.app
Backend	https://paytrace-api.onrender.com (open /health first: the free tier sleeps)
Demo logins	Finance executive and co-founder accounts are listed in deployment.md (repository root)

Throughout this document, IDs such as **(A11)** refer to the labelled assumptions in `docs/ASSUMPTIONS.md`, reproduced in section 9.

1. Problem and scope

Company A is a Bengaluru D2C home-textiles brand selling on its own website, three marketplaces and two quick-commerce apps, shipping through two courier aggregators, with 55% of website orders paid cash on delivery. About 30,000 orders a month at ₹1,200 each pass through partners that deduct commissions, fees and freight before paying out. Roughly 1.5% of order value (₹64.8 lakh a year) leaks through unremitted COD, wrong fees, wrong weight slabs and lost returns. A three-person finance team matches spreadsheets with lookups, and month-end close takes 12 days.

The brief's CORE BUILD: upload partner reports (CSV or Excel) with a column mapping saved per source; order-level matching; rate-card expected amounts flagging gaps over ₹10; a claims list by partner with evidence export; a money-at-risk view by partner and age; two roles (finance executive, co-founder); AI column-mapping suggestions; and matching 1 lakh rows in under a minute. All five statuses from the brief must exist.

What I built: the whole core build, end to end and deployed, plus two of the three optional-depth items: a **returns tracker** against warehouse receipt scans, and an **AI-assisted review queue** for partner lines that arrive without an order ID. The partner-wise TCS/TDS summary exists as a dashboard panel but not as a separate screen.

Success metric (brief)	Target	What the seeded demo shows
Orders auto-matched	98%	**99.0%** of orders where money should have moved (A35); 99.16% in the 1-lakh benchmark
Month-end close	12 days → 3	Not measurable on synthetic data. Every partner file is processed and reconciled in seconds, and claims are prepared automatically, so the manual matching days disappear
Recovered via claims, year one	₹30 lakh	Not measurable yet. One sample month surfaces **₹6.60 lakh** at risk across 1,467 orders in 7 ready-to-file claims

2. Architecture and why

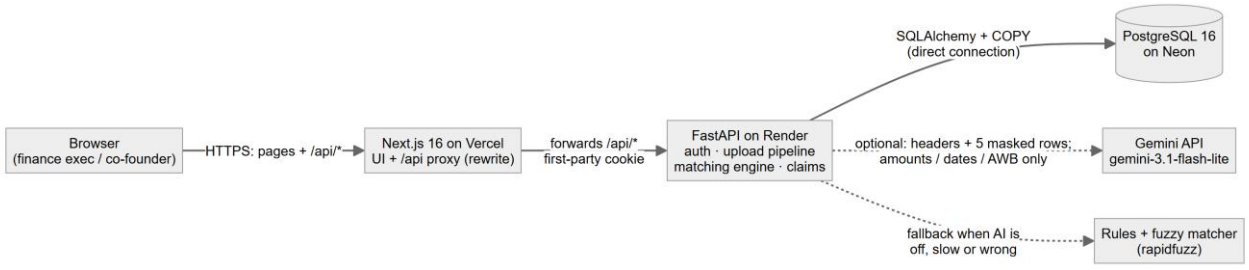


Figure 1. Deployed architecture. Dashed lines are optional; every AI path has a rule-based fallback.

The browser only ever talks to the Next.js site on Vercel. Vercel forwards `/api/*` to the FastAPI service on Render, which owns all business logic and talks to PostgreSQL on Neon. Gemini is called only for the two narrow AI jobs described in section 6.

Choice	Why	Alternative I rejected
FastAPI + Polars in one long-running service (Render)	The heavy step is parsing 2.5 lakh rows and moving them with Postgres COPY. A persistent Python process does that in seconds and keeps typed validation (Pydantic) and a role-check dependency on every route.	A serverless backend: body-size and execution-time limits, cold starts per request, and no place for Polars-sized work.
Proxy mode: Vercel rewrites /api/* to Render	The session cookie becomes first-party, so sign-in works in Safari and on phones that block third-party cookies.	Direct cookie mode (browser calls Render): needs SameSite=None, which Safari and iOS drop. The cost of proxy mode, a ~4.5 MB upload limit, is documented.
Neon direct (non-pooled) connection	The upload pipeline uses temporary tables, and reconciliation takes a session-level advisory lock; both need one stable session.	The pooled (PgBouncer transaction-mode) string, which can hand consecutive statements to different sessions.
PostgreSQL as the only store; money as integer paise	One transactional store; COPY in and out; no float rounding (A10).	A separate analytics store or a queue: unnecessary at 30,000 orders a month.
Gemini behind interfaces with fallbacks	MappingSuggester (LLM or fuzzy) and MatchRanker (LLM or rules). The product works with no key.	Letting AI compute expected amounts or matches: money must be exact and explainable.

3. Data model

Money is stored as integer paise (BIGINT) everywhere and converted to ₹ only for display (A10). Timestamps are UTC. Status-like columns are CHECK-constrained text. Join keys (`order_id`, `awb`, `partner_id`, `status`) are indexed.

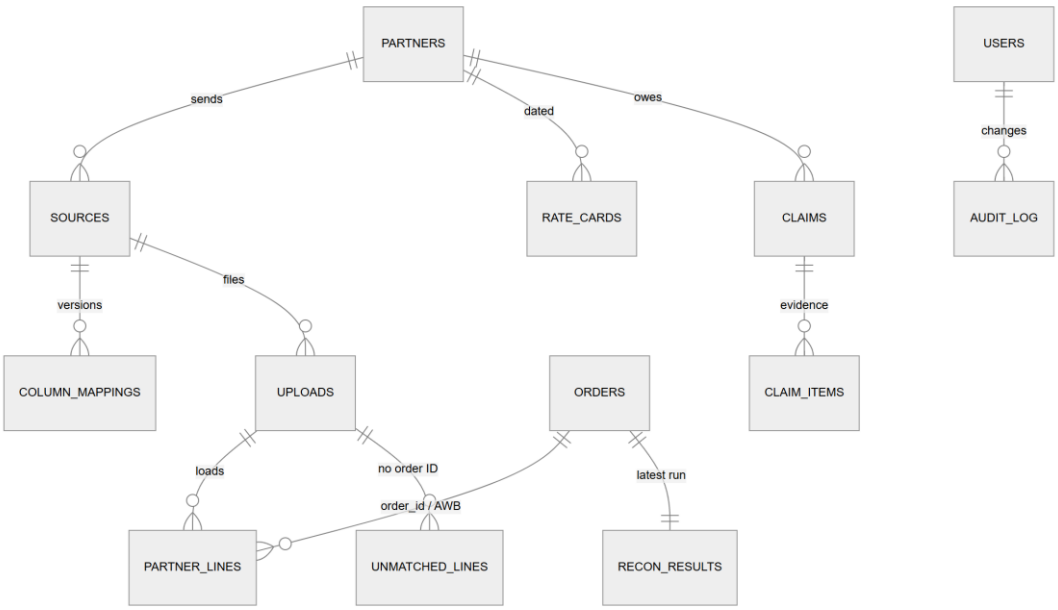


Figure 2. Core entities. "Partner lines" stands for the four line tables.

Table	Purpose	Key fields
users	Two roles	email, bcrypt password_hash, role (finance_exec | cofounder)

Table	Purpose	Key fields
partners, sources	Who sends what	partner kind and settlement_days (A3); source = partner + file kind
column_mappings	Versioned mappings per source	version, mapping_json, header_fingerprint, is_active, ai_suggested_json, approved_at
uploads, upload_row_errors	Every file and every rejected row	row counts (ok / rejected / duplicate / queued), duration_ms; row_number + reason
orders	One row per order	order_id, channel and courier, awb, dates, value_paise, payment_mode, delivery_status, weights, **customer_name_masked**, **customer_phone_masked**, pincode
settlement_lines, cod_remittances, freight_lines, warehouse_receipts	Partner lines (high volume)	amounts in paise, line_hash (md5 of canonical values: no double counting, A26), row_number for evidence
rate_cards, weight_slabs	Effective-dated terms	commission/gateway/COD/TCS/TDS in basis points, fixed fees in paise; slabs per leg
recon_runs, recon_results	Reconciliation	expected/received/diff/at-risk paise, tcs/tds kept apart, breakdown and evidence JSON, status, due_date, age_days
claims, claim_items	Claims per partner	status open → filed → recovered/rejected (A7); evidence snapshot per item
unmatched_lines, manual_return_receipts	Review queue; returns confirmed by hand	candidates, suggestion, confidence, engine, decision; note + who + when
audit_log	Who changed what	mapping approvals and rollbacks, rate-card edits, claim status, queue decisions, manual receipts (before/after JSON)

Personal data (DPDP Act 2023). Names keep only their first and last letters and phones only their first and last two digits, masked **at ingestion**: the raw values are never stored. Addresses are dropped; only the pincode is kept (A28).

4. Main flows

4.1 From upload to claim

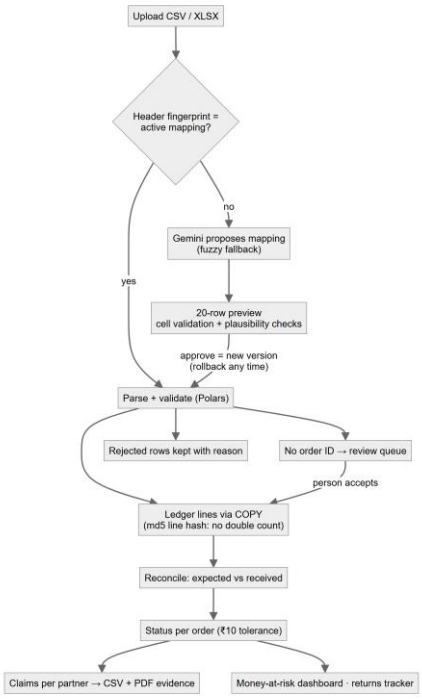


Figure 3. Upload to claim.

1. **Upload** a .csv or .xlsx for a chosen source (validated type, 50 MB limit; about 4 MB through the proxy).
2. **Fingerprint** the normalised, sorted headers. If they match the active mapping, it is reused silently ("Using saved mapping vN"). If they match an older version, that version is proposed but needs approval. Otherwise the AI proposes a mapping (A29).
3. **Preview 20 rows** with per-cell validation (dates, amounts, order IDs, required fields) and rule-based plausibility checks (A40). Low-confidence or failing fields must be confirmed.
4. **Approve** saves the mapping as a new version; **rollback** re-activates any earlier version. Both are audited.
5. **Ingest** with Polars and COPY into a temporary table, then one INSERT…SELECT with md5 line hashes, so a re-sent report is recognised and never double-counted (A26). Rejected rows are kept with a reason; lines with no order ID go to the review queue (A43).
6. **Reconcile** every order, assign a **status**, refresh **claims** per owing partner, and **export** evidence as CSV (every figure traced to file and row) and a PDF summary.

4.2 Expected amount, with the brief's three sample orders

```
delivered:  expected = value − commission − fixed fee − gateway fee (prepaid)
                             − COD fee (COD) − forward freight
returned:   expected = −(forward freight + return freight)     channel fees reversed (A15)
freight:    slab price for max(dead, volumetric weight)        top slab if heavier (A9, A16)
received:   settlement net + TCS + TDS + COD remitted − courier charges invoiced (A18)
difference: received − expected                  TCS/TDS never in the difference (A6)
```

Order	Facts	Expected	Received	Diff	Status
ORD-W-204118	Website, ₹1,499 COD, Courier A, 450 g	1,499 − COD fee 30 − freight 60 = **₹1,409**	₹0: COD collected, never remitted	−₹1,409	unpaid_past_due
ORD-M-88120	Marketplace 1, ₹1,199 prepaid	1,199 − commission 20% (239.80) − fixed fee 60.20 = **₹899**	net 854.80 + TCS 6.00 + TDS 1.20 = **₹862**	−₹37	short_paid
ORD-W-204377	Website, ₹2,398 prepaid, returned, Courier B, 480 g	−(forward 48 + return 48) = **−₹96**	+2,350.04 − 2,350.04 refund − freight billed on 900 g (71 + 71) = **−₹142**	−₹46	freight_overcharged

The seed rate cards are assumptions chosen to reproduce these rows exactly (A17); tests assert 1,409 / 899 / −96.

4.3 Status rules (first match wins) and the ₹10 tolerance

#	Status	Rule
1	return_not_received	Returned or RTO, no warehouse receipt, and the 7-day receipt window has passed (A8, A23, A31)
2	freight_overcharged	Courier freight charged − expected freight > ₹10
3	unpaid_past_due	Delivered, nothing received, and the paying partner's due date has passed (A3, A22, A33)
4	pending	In transit, not yet due, or a return still inside its window: not money at risk (A11)

#	Status	Rule
5	short_paid	Received < expected − ₹10
6	settled	Everything else: within ₹10, or overpaid by more than ₹10 (flagged, A12)

The ₹10 tolerance applies per order to the net difference: exactly ₹10.00 is settled, ₹10.01 is flagged (A5; tested at the boundary).

4.4 Returns tracker and review queue

- **Returns tracker:** every returned/RTO order is received (scan, or confirmed by hand), waiting (inside the window) or not received (A42). The not-received set is by construction exactly the return_not_received results. A finance executive can mark a return received with a required note (A44); it is audited and the order is re-matched at once.
- **Review queue:** lines with no order ID are shortlisted by rules and ranked by Gemini (section 6). Accepting writes the line to the ledger with the hash it would have had with an order ID, re-matches only that order, refreshes claims and writes the audit log. Nothing is ever linked automatically.

5. The hard part: 1 lakh rows in under a minute

`scripts/benchmark_matching.py` generates 100,000 orders and 2,55,199 partner rows in every partner's own file layout, with ~1.5% injected leakage, and runs the real pipeline (parse → normalise → COPY → match → COPY results) against a separate database. Latest run, on a laptop with Postgres in Docker:

Stage	Time
Parse + normalise + COPY all 17 partner files (2,55,199 rows)	9.61 s
Reconciliation: load from Postgres (COPY TO STDOUT)	0.54 s
Reconciliation: match (Polars joins, expected amounts, status rules)	0.20 s
Reconciliation: write 100,000 results (COPY)	3.16 s
End to end	**13.57 s** (target < 60 s)

Earlier runs on the same machine took 18.2 s, and 21.5 s as the slow pytest. Detection: 4,262 of 4,262 due leaks flagged, 0 false alarms, 99.16% auto-matched.

- **No Python loop per row.** Files are typed with vectorised Polars expressions. Matching uses hash joins (orders ⋈ aggregated lines by order ID, with AWB fallback, A24) and as-of joins (the rate card in force on the order date; the weight slab covering the chargeable weight). Status rules are a single when/then expression.
- **COPY both ways.** Lines enter through COPY into a temporary table and one INSERT…SELECT; results are read with COPY TO STDOUT and written back with COPY inside one transaction (TRUNCATE + COPY, so readers never see a half-written run, A36).
- **Readable reference engine.** `expected.py` is plain Python; a test runs it and the Polars engine on 6,000 orders and requires identical amounts.

The foreign-key decision. Postgres checks foreign keys row by row; with them on the bulk tables the run took 31 s, without them 18 s at the time. I removed them from orders, the four line tables and recon_results only. Integrity is still kept: only the ingest pipeline and the engine write those tables, every partner, upload and run ID they write comes from a record they have just validated in the same transaction, and the low-volume tables (users, partners, sources, mappings, uploads, rate cards, claims, audit, review queue) keep their foreign keys.

6. How the AI works, and what happens when it is wrong or unsure

Matching and expected amounts are rules, never AI. The AI does two narrow jobs, each fenced by rules and a person.

6.1 Column mapping

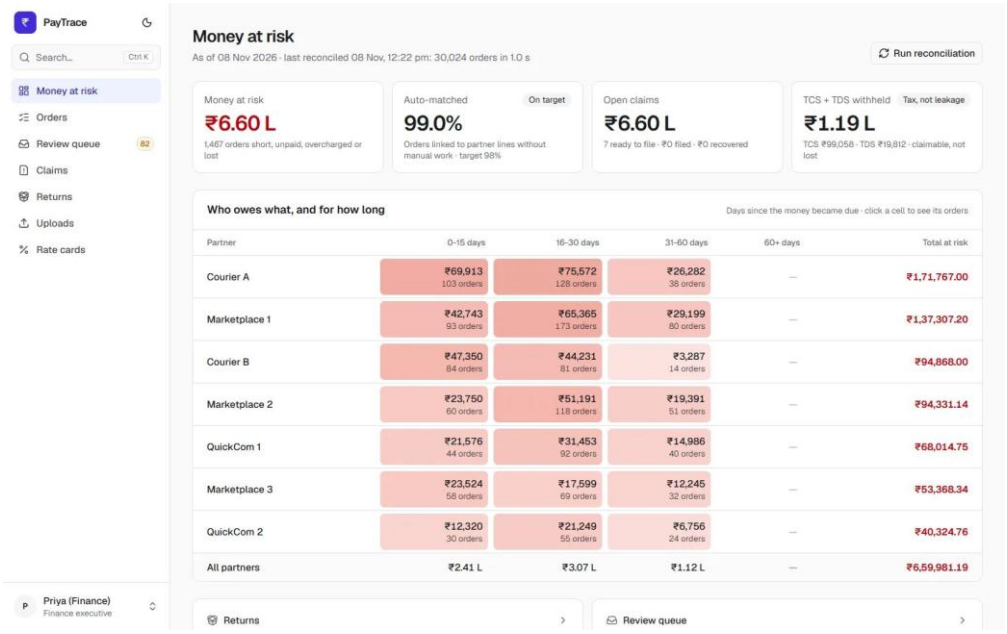
- **Input:** only the headers and 5 sample rows, with name-, phone- and address-like columns masked before sending. **Output:** JSON per field with source column, confidence (0–1) and a one-line reason, validated with Pydantic. Model: `gemini-3.1-flash-lite`, configurable.
- **Safety net:** the 20-row preview with cell validation; fields under 0.7 confidence or with errors must be confirmed; only Approve saves, as a new version; rollback any time. On no key, timeout, HTTP error or invalid JSON, a fuzzy matcher (rapidfuzz on header synonyms plus value types) takes over and the screen says "AI unavailable, showing best guesses".
- **Measured** (`scripts/eval_ai_mapping.py`, 18 sample layouts + 2 deliberately hard ones): Gemini **194/194** fields right, median 2.6 s per file; fuzzy fallback 190/194.
- **What testing with a real key exposed.** (1) The first real run silently used the fuzzy fallback for every file: `gemini-2.5-flash` returns 404 to new API keys. The fallback worked as designed, but the AI was never used. I made the model configurable and switched after comparing models. (2) On a cryptic layout the model mapped `Tax A` → TCS and `Tax B` → TDS, swapped, at confidence 0.70: just above the review threshold, with every cell parsing. Nothing would have flagged it. I added rule-based plausibility checks on the preview (TCS should exceed TDS; gross − deductions = net, A40). The swap is now flagged on both fields and cannot be approved without confirmation. I also corrected the prompt.

6.2 Suggested matches for lines without an order ID

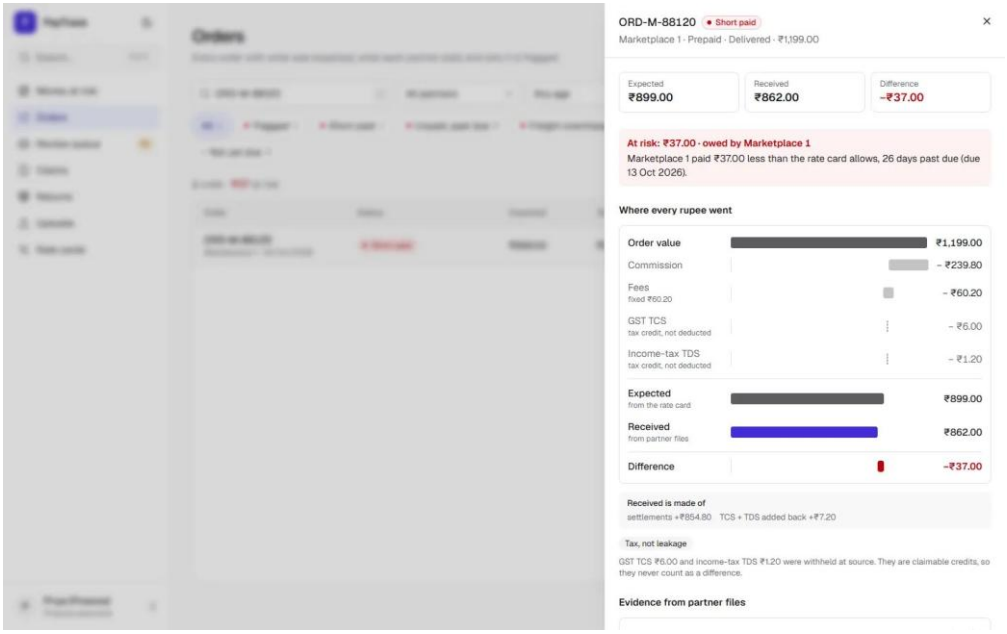
- **Step 1, rules (deterministic, vectorised):** up to five candidates per line by partner, unpaid status, amount within ₹10 or 2%, a date window around the paying partner's due date, or an exact AWB (A45). Rule confidence = top score − half the runner-up's, so look-alikes never look certain.
- **Step 2, Gemini:** sees amounts, dates, AWB and partner only, never customer data (tested). It ranks the shortlist with a confidence and a reason. Any failure falls back to the rule order, with the reason shown.
- **Measured, then designed** (`scripts/eval_match_suggestions.py`, 82 lines with known answers): the rules shortlisted the right order **82/82** and suggested it **61/82**. The other 21 are exact ties (same partner, value and due date) that no data can separate. Gemini choosing freely got only **47–51/82** and was confident and wrong on 5 lines. So in the final design it breaks exact ties and explains, never overrules a clear rule winner, and its confidence is **capped by the rule evidence** (A46). Result: 61/82 right and **0 wrong at the 0.9 bulk-accept threshold**. The evaluation also caught a bug: COD candidates' due dates used the sales channel's cycle instead of the courier's.
- **Never auto-closed:** a line is linked only when a finance executive clicks Accept; bulk accept is limited to confidence ≥ 0.9 and still needs a confirmation.

7. UI and design decisions

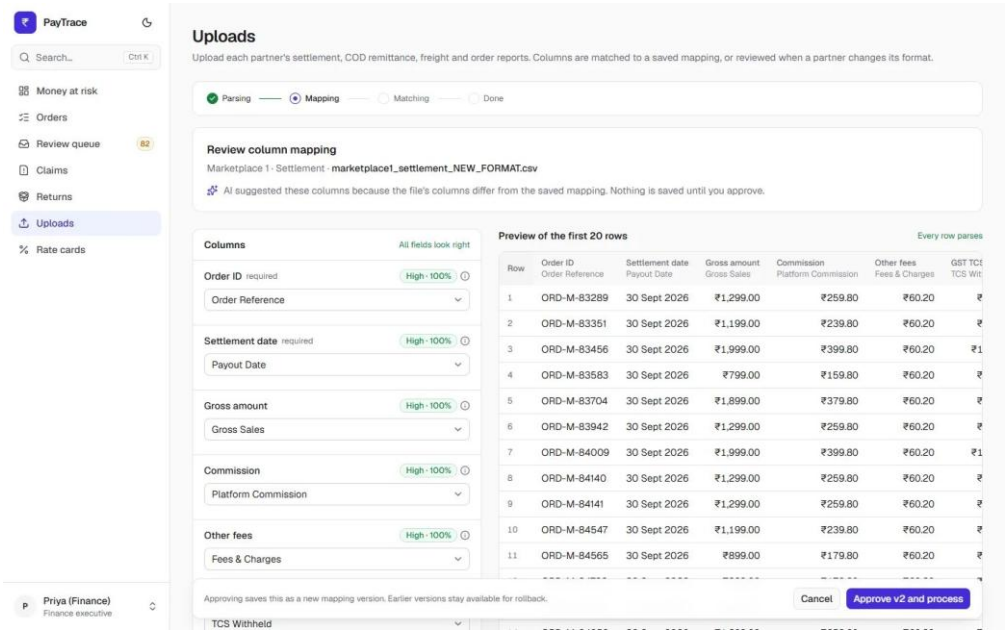
The brief asks for minimal, purposeful design. I used near-white surfaces, zinc neutrals and **one indigo accent**. Colour carries meaning only: red is money lost or at risk, amber needs attention, green is settled. Money uses tabular numerals and Indian grouping (₹1,23,456.00). Every screen has loading, empty and error states.



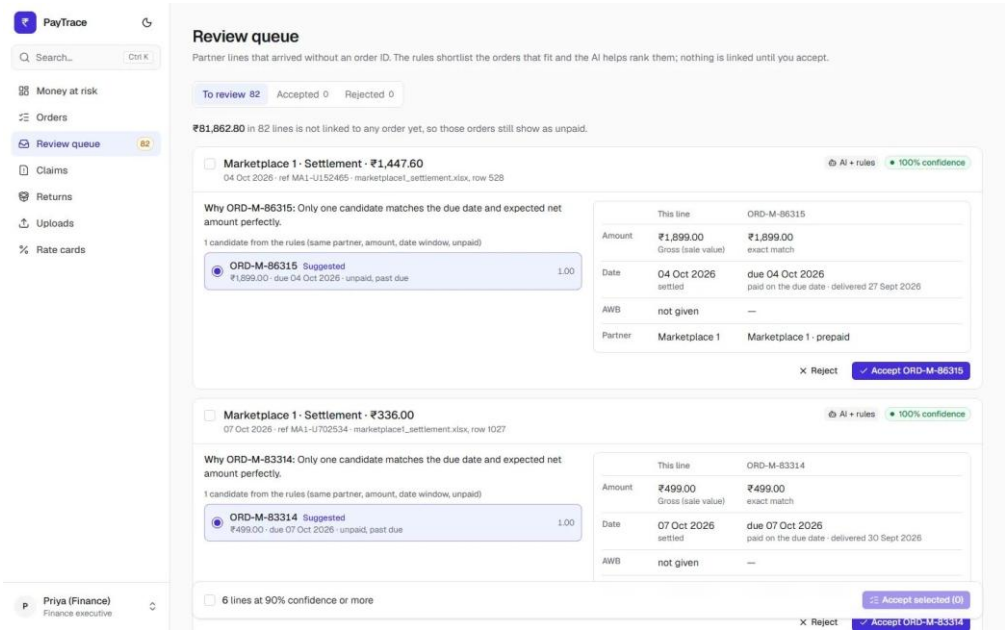
Money at risk. Four numbers, then a partner × age matrix whose heat tint makes the biggest, oldest debts stand out. Every cell drills into exactly those orders. Tax withheld has its own card labelled "Tax, not leakage".



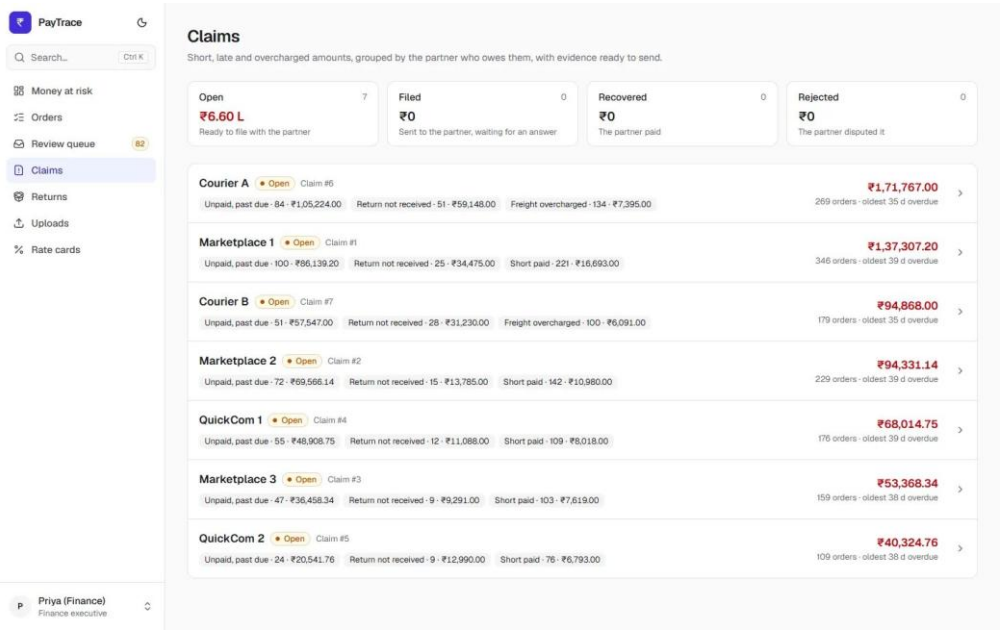
Order detail for the brief's ORD-M-88120. "Where every rupee went" itemises the rate card, shows TCS/TDS as credits that are never deducted, and turns red only for a shortfall beyond ₹10. Evidence lines cite file and row.



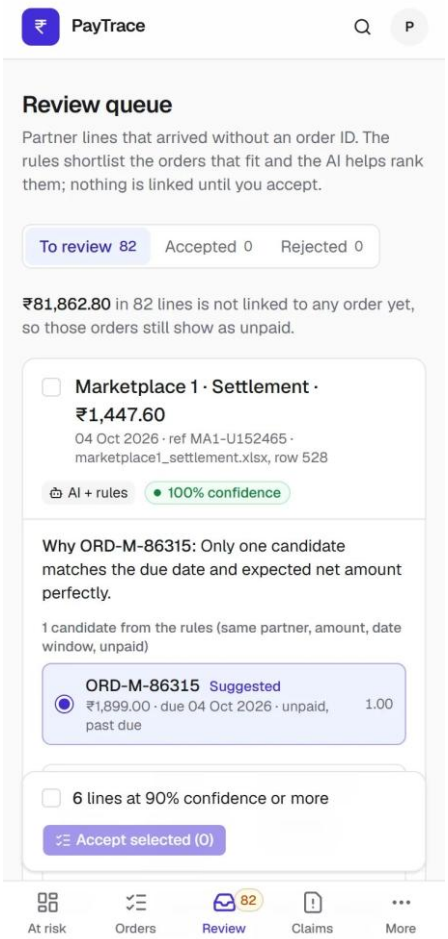
Mapping review. Field → source column with confidence and reason on the left, the 20 parsed rows on the right, and an Approve bar that states a new version will be saved and older ones kept for rollback.



Review queue. Each line shows its suggestion, confidence and engine, the reason, the candidates and a side-by-side comparison that follows the selected candidate. Bulk accept is offered only for lines at 90% or more.



Claims. Status cards double as filters; each partner row summarises why money is owed. Tables and dense lists suit a finance team; filters stay to a few chips with counts.



Phone (375 px). Tables become cards, drawers become bottom sheets, controls are at least 44 px, and a five-target bottom bar (with the queue count) keeps every screen one tap away.

8. Security, privacy and testing

- **Roles:** finance executive (everything) and co-founder (read and export only, A2). Controls the co-founder cannot use are hidden, not disabled, and every write route requires the finance role on the

server; the role is read from the database on each request. A test sweeps the whole route table and fails if any write route lacks the guard; live checks confirmed 403s.

- **Authentication:** bcrypt hashes; JWT in an httpOnly cookie (8-hour sessions, A13); unknown emails take the same time as wrong passwords; failed logins are rate-limited to 5 per minute per client IP (A14), honouring X-Forwarded-For only for configured proxies.
- **Validation:** Pydantic on every body and query; uploads limited to .csv/.xlsx and 50 MB.
- **Secrets:** all configuration from environment variables; `.env*` git-ignored; `.env.example` lists names only; the API refuses to start without a 32+ character JWT secret; the Gemini key is sent in a header and never logged. Git history was scanned before publishing.
- **Exports:** CSV cells starting with =, +, -, @ are prefixed to stop formula injection (plain numbers untouched). PDF uses "Rs" because its built-in fonts lack ₹ (A39).
- **Headers and CORS:** nosniff, X-Frame-Options DENY, strict referrer policy, Cache-Control no-store; CORS restricted to the frontend origin; API docs off in production.
- **Privacy:** masking at ingestion, addresses never stored, no unmasked PII to the AI (section 3, tested).

Tests (pytest)	What they prove
337 total: 336 regular + 1 slow	All pass on the final code.
Engines	The three brief samples (1,409 / 899 / −96); every status rule including the ₹10 boundary; TCS/TDS never in the difference; multiple and duplicate lines; reference and vectorised engines agree on 6,000 orders; every injected leak caught with no false alarms.
Mapping	Fingerprint reuse; invalid AI JSON → fuzzy fallback; validation catches bad dates and amounts; plausibility checks; approve and rollback; no raw PII sent.
Returns (15) and review queue (24)	Scan-window boundary; tracker agrees with matching; manual receipt clears the claim; shortlist rules; AI fallback on bad answers; no auto-close; accept, reject, bulk; access.
Access and performance	Co-founder blocked on every write route; the 1-lakh slow test must finish under 60 s (21.5 s).

Frontend: ESLint and the TypeScript production build pass. Lighthouse (desktop): dashboard 100/100/100 and orders 99/100/100 for performance, accessibility and best practices.

9. Assumptions

Full wording is in `docs/ASSUMPTIONS.md`. **CA** marks items to confirm with the client's chartered accountant; **verify** marks items to confirm with the client.

ID	Assumption
A1	Partner names are generic (Website, Marketplace 1–3, QuickCom 1–2, Courier A/B).
A2	Co-founder is read-only: views and exports, no uploads, mappings, rate cards or claim changes.
A3	Settlement cycles: marketplaces and quick-commerce T+7, courier COD T+10; configurable. **verify**
A4	Money-at-risk age buckets 0–15, 16–30, 31–60, 60+ days past due.
A5	₹10 tolerance per order on the net difference; ₹10.00 settled, ₹10.01 flagged.
A6	TCS 0.5% and TDS 0.1% by default, configurable, shown separately, never leakage. **CA**
A7	Claims move open → filed → recovered/rejected by hand; filing happens outside the app.
A8	Return not received = returned/RTO with no warehouse scan within 7 days.
A9	Freight by weight slab on max(dead, volumetric) weight, per courier and leg.
A10	Money in integer paise; ₹ with 2 decimals in Indian grouping.
A11	Not-yet-due orders are pending and not money at risk.
A12	Overpaid by more than ₹10 stays settled with an overpaid flag.
A13	8-hour sessions, no refresh tokens.
A14	Login rate limit: 5 failed attempts per minute per IP and email, in memory.
A15	Returns reverse the sale and channel fees; expected = −(both freight legs). **verify**
A16	Heavier than the top slab → top-slab price.
A17	Seed rate cards are illustrative and reproduce the brief's three orders exactly.
A18	Received = settlement net + TCS + TDS + COD remitted − courier charges.

ID	Assumption
A19	Percentages rounded half up to the paisa per line item.
A20	TCS/TDS computed on order value including GST (no GST split in the brief). **CA**
A21	Marketplace/quick-commerce orders use the channel's logistics; only website orders use couriers.
A22	Payment gateway settles prepaid website orders T+3.
A23	Receipt window counts from the courier's returned date; configurable.
A24	Partners report the brand's order ID; couriers may report only the AWB.
A25	Sample month: 30,000 orders, realistic mix, ~1.6% injected leakage, invented customers.
A26	Identical partner lines are duplicates, never double-counted; orders upsert by ID.
A27	Dates read day-first; weight in kg if the header says kg, else grams.
A28	Addresses not stored; names and phones stored masked.
A29	Matching fingerprint → reuse; older version → propose; else AI + 20-row review.
A30	Uploaded files wait at most 24 hours on disk, then are deleted.
A31	A late scan still counts as received; no scan after the window = not received.
A32	Owing partner: courier for COD, freight and lost returns it carried; else the channel.
A33	Due date = paying partner's cycle; window end for returns; courier cycle for freight.
A34	Money at risk = shortfall; goods value + shortfall for a lost return.
A35	Auto-match rate over orders where money should have moved.
A36	Only the latest run's per-order results are kept; every run's stats are kept.
A37	An order is claimed at most once.
A38	One open claim per partner; filing freezes it.
A39	PDF uses "Rs" instead of ₹ (built-in fonts).
A40	TCS > TDS and gross − deductions = net are used as preview plausibility checks. **CA**
A41	Recovered claims lower money at risk only when the payment arrives in a file.
A42	Returns tracker states: received, waiting, not received; ages from the latest run.
A43	Lines with no order ID go to the review queue unless over 20% of a file lacks IDs.
A44	A return can be confirmed by hand with a required, audited note.
A45	Rule shortlist: partner, unpaid, amount ₹10/2%, date window, AWB; pincode not available.
A46	AI breaks ties and explains; never overrules a clear rule winner; confidence capped.
A47	The seed adds recent returns so all tracker states show; they age out in 7 days.

10. How I used AI

I used AI tools throughout. I chose the problem, set the scope and made the design decisions; I verified results by running tests, benchmarks and real-browser click-throughs rather than trusting a tool's word.

- **Claude (chat):** compared the five problem statements, helped me pick this one, and helped me write the spec (scope, data model, matching rules, assumptions).
- **Claude Code:** implemented the code milestone by milestone from my spec, wrote tests, and ran checks and browser click-throughs. Its commits carry a Co-Authored-By trailer.
- **Gemini API:** the in-product AI for column-mapping suggestions and for tie-breaking and explaining review-queue suggestions (confidence capped by rules), measured with two eval scripts.

What I checked and decided myself: testing with a real key (which exposed the silent 404 fallback), the TCS/TDS swap and the rule checks added for it, proxy cookie mode for Safari and phones, dropping foreign keys on the bulk tables while keeping integrity, and keeping matching and expected amounts as rules.

11. What I cut and what I'd do next

Honest limits of what is shipped:

- **Render free tier cold start:** the backend sleeps after 15 idle minutes; the first request can take about a minute. Reviewers should open /health first.
- **About 4 MB upload limit through the Vercel proxy.** A month of this client's reports fits; larger files must be split, or both apps moved onto one domain to upload directly (50 MB).
- **Mobile Lighthouse performance 77–79** (desktop 99–100): every screen renders client-side after a session check.
- **No Tally export,** and **no automatic claim filing** (explicitly out of scope; claims are prepared, filing is manual, A7).
- **Pincode is not a matching signal:** no partner's settlement or remittance layout carries one (A45). 21 of 82 queue lines are genuine ties that need a person.
- **No returns-manager role:** returns are handled by the finance executive; the brief names only two roles.
- Background AI ranking runs in-process; the login limiter is in memory (single instance); users are seeded, with no user-management screen.

Next steps, in priority order:

1. Continuous integration running the 337 tests, lint and build on every push.
2. Tally export of reconciled settlements and claim recoveries, so close happens in the books the client already uses.
3. Server-render the first screen (now possible with the first-party cookie) to lift mobile performance above 90.
4. Move AI ranking and long uploads to a persistent job queue; add the bank UTR as a matching signal.
5. Partner-wise TCS/TDS reconciliation against Form 26AS, confirmed with the client's CA (A6, A20).
6. Paid hosting or a shared domain to remove the cold start and the proxy upload limit; user management and per-user audit views.